

1 OS structures

NOTE:

Throughout the exam, "the company" refers to the hypothetical company introduced in the first question. Throughout the exam, "the designed OS" or "the OS" refers to the hypothetical Unix-like OS introduced in the first question.

A start-up tech company is designing and implementing its own operating system for a specific device. They aim to have a minimal kernel space for their OS, which type of OS structure should they select?

Select one alternative:

- ☐ Layered kernel
- ☐ Microkernel
- ☐ Monolithic Kernel
- ☐ Hybridkernel

The company plans to design a Unix-like OS that is divided into user mode/space and kernel mode/space. In such Unix-like OSs, what is a system call?

Select one alternative

- ☐ A hardware instruction for process synchronization
- ☐ A command executed in a shell terminal
- ☐ A request made by a user program to the operating system kernel
- ☐ A function provided by a programming language's standard library

Maximum marks: 3

2 Process and Thread

The company intends to use the abstraction of processes as in Unix-like OSs. Which of the following is **TRUE** about processes?

Select one alternative:

- ☐ A program stored on the disk
- ☐ A system call
- ☐ A program in execution
- ☐ A block of memory

The new OS implements a data structure -- the process control block -- to store all relevant information of a process. Which of the following is **NOT** part of a process's Process Control Block (PCB)? Hint: Think about the struct proc in xv6-riscv.

Select one alternative

- ☐ Disk size
- ☐ Program counter
- ☐ Process ID
- ☐ Page table

Which system call is used to create a process in Unix-like systems?

Select one alternative

- ☐ pthread_create()
- ☐ createprocess()
- ☐ fork()
- ☐ exec()

What is the purpose of the exec() system call?

Select one alternative

- ☐ To terminate a process
- ☐ To create a child process
- ☐ To create a new process with a new program
- ☐ To replace the current process with a new program

The target device has a multi-core CPU, so the company also implements threads as in Unix-like OSs. What is the fundamental difference between a process and a thread?

Select one alternative

- ☐ A process shares memory with other processes, while a thread does not.
- ☐ A process is managed by the CPU, while a thread is managed by the user.
- ☐ A process is a program in execution, while a thread is a subset of a process.
- ☐ A thread has its own address space, while a process does not.

Which of the following is shared among threads of the same process?

Select one alternative

- ☐ Register values
- ☐ Stack memory
- ☐ Program counter
- ☐ Global variables

Maximum marks: 9

3 Scheduling

The engineers from the tech company need to implement a scheduling algorithm to manage processes or threads on the system. It first evaluates several scheduling algorithms to determine the best option for their system.

Assuming that the execution times of tasks are known in advance, which **non-preemptive** scheduling algorithm is optimal for minimizing the average waiting time ?

Select one alternative:

- ☐ STCF
- ☐ FIFO
- ☐ Round-Robin
- ☐ SJF

SJF requires prior knowledge of execution time, which is often difficult to obtain in practice. The engineers in the company then evaluate round-robin (RR) scheduling as an alternative. If the time slice/quantum in RR scheduling is set to a very large value, how will it behave?

Select one alternative

- ☐ Like STCF
- ☐ Like FIFO
- ☐ Like SJF
- ☐ Like priority scheduling

If the time slice is too small in RR, what problem occurs?

Select one alternative

- ☐ Starvation of long processes
- ☐ Deadlocks
- ☐ High context-switching overhead
- ☐ Increased average waiting time

For which type of processes does RR scheduling typically provide the best improvement over FIFO and STCF in terms of average response time?

Select one alternative

- ☐ Interactive processes
- ☐ Processes executed in kernel mode
- ☐ Processes with high I/O demands
- ☐ Processes with long execution time

4 Memory

The designed OS uses virtual memory to manage physical memory on the device. Like other modern computing systems, it relies on specific hardware to manage virtual memory. Which hardware component is responsible for translating virtual addresses to physical addresses?

Select one alternative:

- ☐ Memory management unit
- ☐ Memory scheduler
- ☐ Memory mapper
- ☐ Memory controller

The company considers using the dynamic relocation method to manage virtual memory. Which of the following is **TRUE** about the dynamic relocation method?

Select one alternative

- ☐ It adjusts memory addresses of a process at load time.
- ☐ It determines the physical addresses of a process during execution.
- ☐ It uses a limit/bound register to store the ending address of a process.
- ☐ It allocates fixed memory blocks to processes.

Managing memory in a continuous manner can lead to memory underutilization, so memory segmentation is used in some OSs to improve memory utilization. What is memory segmentation in OS?

Select one alternative

- ☐ Splitting memory into variable-sized logical units
- ☐ Dividing memory into fixed-size blocks
- ☐ Allocating an entire process's memory to a segmentation
- ☐ Splitting memory into pages

When allocating a memory space to a segment, different memory allocation algorithms can be used. Which memory allocation algorithm assigns the smallest available partition that is large enough for the process?

Select one alternative

- ☐ Worst-Fit
- ☐ Next-Fit
- ☐ First-Fit
- ☐ Best-Fit

5 Paging

Memory segmentation suffers from external fragmentation and is difficult to manage, so the company decides to use paging to manage virtual memory. Which of the following is **TRUE** about paging?

1. Paging can lead to internal fragmentation
2. Each page in a virtual address space is mapped to multiple frames in physical memory
3. Pages and frames in a system have different sizes
4. In paging, translation information is stored in a page table

Select one alternative:

- ☐ 1,3
- ☐ 1,4
- ☐ 2,4
- ☐ 2,3

The company first evaluates the inverted paging method for their OS. Which of the following is **TRUE** about inverted paging?

1. Inverted paging reduces the size of the page table by maintaining a single entry per physical frame
2. Inverted paging can support page sharing
3. An entry in the inverted page table stores a page number and process ID
4. Inverted paging is faster than traditional paging

Select one alternative

- ☐ 1,3
- ☐ 2,4
- ☐ 1,2
- ☐ 2,3

After evaluating inverted paging, they finally decide to use traditional paging, which uses a separate page table for each process. In this paging method, the virtual address is divided into:

Select one alternative

- ☐ Frame number and offset
- ☐ Base and limit
- ☐ Segment number and offset
- ☐ Page number and offset

HINT: In our context, the terms "page number" and "page index" refer to the same concept.

The target device has a 64-bit CPU and the default page size is set to 16KB. How many bits are used for the offset?

Select one alternative

- ☐ 12 bits
- ☐ 14 bits
- ☐ 15 bits
- ☐ 13 bits

It is found that a larger page size is more likely to result in wasted memory due to internal fragmentation, so the OS reduces the page size to 4KB. Additionally, it implements a linear page table and uses only 39 bits for virtual addresses. How many bits are used for the page number?

Select one alternative

- ☐ 12
- ☐ 28
- ☐ 27
- ☐ 24

When a process is initialized, its page table is loaded into main memory. As a result, each memory access requires two accesses to main memory. A small cache, called the translation lookaside buffer (TLB), is added to accelerate the translation of virtual to physical addresses. Which of the following is **TRUE** about TLB?

Select one alternative

- ☐ The TLB is a software-managed cache.
- ☐ The TLB typically stores the entire page table of one process.
- ☐ The TLB typically stores some frequently accessed pages for the currently running processes.
- ☐ A TLB miss occurs when the page table entry is not found in the TLB.

If the TLB is full, a page replacement algorithm is used to replace some of the entries in the TLB. Which of the following is not a page replacement algorithm?

Select one alternative

- ☐ FIFO
- ☐ Least-recently used
- ☐ Round-Robin
- ☐ RANDOM

In addition to slow address translation, a linear paging table also results in a larger page table size. It is recommended to implement a multi-level page table. If each page table entry takes 4 bytes, how many entries can

fit in a single page of 4KB? The OS implements a 3-level page table. How many bits are used for each level of the page directory?

Select one alternative

- ☐ 1024, 10 bits
- ☐ 512, 11 bits
- ☐ 512, 8 bits
- ☐ 1024, 9 bits

Maximum marks: 12

6 Concurrency

There are two processes accessing a shared counter **X**, which is initialized to **1**. Each time, Process 1 increments X by 1, while Process 2 decrements X by 1. Below are the instructions used by Process 1 and Process 2 to increment and decrement the counter, respectively.

Process 1	Process 2
load R1, X //load X to register	load R2, X
inc R1	dec R2
store X, R1 //store X to register	store X, R2

After executing Process 1 and Process 2 once each, what are the possible values of X?

Select one alternative:

- ☐ -1, 3
- ☐ -1, 0, 1, 2
- ☐ 0, 1, 2
- ☐ 1

What is the main role of a condition variable in addressing concurrent issues?

Select one alternative

- ☐ To signal threads to wake up when a specific condition is true
- ☐ To prevent deadlock by limiting the number of threads in a critical section
- ☐ To provide mutual exclusion among threads accessing a shared variable
- ☐ To protect shared data from being corrupted by multiple threads

Which of the following conditions must hold simultaneously for a deadlock to occur?

Select one alternative

- ☐ Preemption, Mutual Exclusion, Resource Sharing, Circular wait
- ☐ Mutual Exclusion, Hold and Wait, Preemption, Resource Sharing
- ☐ No Preemption, Mutual Exclusion, Hold and Wait, Resource Sharing
- ☐ Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait

Maximum marks: 4.5

7 I/O devices

What is the advantage of Direct Memory Access (DMA)?

Select one alternative:

- ☐ Increases CPU utilization by polling devices constantly
- ☐ Requires no device drivers
- ☐ Reduces CPU overhead by allowing devices to transfer data directly to memory
- ☐ Slows down I/O operations for better synchronization

In memory-mapped I/O, how does the CPU interact with I/O devices?

Select one alternative

- ☐ By using special I/O instructions to access separate I/O ports
- ☐ By relying on the DMA to send I/O requests to devices
- ☐ By mapping device registers into the memory address space and accessing them with standard memory instructions
- ☐ By using interrupts to access I/O devices

Which disk scheduling algorithm reorders requests to minimize disk head movement?

Select one alternative

- ☐ FIFO (First-In-First-Out)
- ☐ SSTF (Shortest Seek Time First)
- ☐ Priority scheduling
- ☐ Round-Robin

What are the primary storage media used in an HDD and an SSD, respectively?

1. NAND flash memory
2. Magnetic platters
3. Optical discs
4. Volatile RAM

Select one alternative

- ☐ 3, 1
- ☐ 3, 2
- ☐ 2, 4
- ☐ 2, 1

A 7200 RPM hard disk drive (HDD) receives a request to read a block of data from a random location on the disk.

What is the average total latency (in milliseconds) for this operation?

Assume:

- **Average seek time:** 9 ms
- **Rotational speed:** 7200 RPM
- **Transfer time:** negligible (0 ms)

(Note: how to calculate the average rotation latency)

Select one alternative

- ☐ 13.17
- ☐ 17.33
- ☐ 9
- ☐ 4.17

Maximum marks: 7.5

8 Files

Which of the following correctly matches three distinct ways an OS refers to a file?

Select one alternative:

- ☐ Path, inode, file descriptor
- ☐ inode, file pointer, file descriptor
- ☐ file name, path, file descriptor
- ☐ File name, file pointer, inode

Inode is a data structure to represent a file in OS, and it includes the metadata of the file. Which of the following is **NOT** stored in an inode?

Select one alternative

- ☐ Filename
- ☐ Timestamps (creation, modification)
- ☐ File ownership
- ☐ File Size

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("example.txt", O_RDONLY); // Open file for reading only

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // File is opened, now can perform read/write operations
    close(fd); // Close the file
    return 0;
}
```

The above code snippet opens a file called "example.txt" using the system call **open()**. What does the variable **fd** represent?

Select one alternative

- ☐ Link
- ☐ File descriptor
- ☐ Inode
- ☐ Path

If a process opens a file using the **open()** system call, the return value of **open()** is 5. How many files has been opened by the process **before this call**?

Select one alternative

- ☐ 2
- ☐ 5
- ☐ 4
- ☐ 3

What is the role of the superblock in a file system?

Select one alternative

- ☐ It lists all files stored in the disk
- ☐ It maps physical memory to disk
- ☐ It stores user permissions for files
- ☐ It contains metadata about the file system layout and status

In a file system similar to the **very simple file system** discussed in the lecture and the textbook, how is the free space managed?

Select one alternative

- ☐ Through the inode table
- ☐ Through a linked list of available blocks
- ☐ With a dynamic memory allocator
- ☐ Using a bitmap where each bit represents a block's availability

Maximum marks: 9

9 Process

Each blank is worth 2 points.

The OS has a shell application, a command-line interpreter widely used in Unix-like OSs which is used to execute a new command.

Since the OS is Unix-like, it uses the three system calls, `fork()`, `exec()`, and `wait()` to manage processes.

The shell process is considered the parent process, and executing a new command involves creating a child process from the parent.

The shell must wait for that new command to complete before proceeding.

First, fill in the blanks with the three system calls, **`fork()`**, **`exec()`**, and **`wait()`**, in the following code snippet to complete the shell application. Arguments for the `exec()` function can be skipped.

// indicates a comment

Shell Process:

```
int main() {
    int count = 0;
    while (TRUE) {
        if (gets () == NULL) // gets() is a function used to receive input the new command.
            continue;
        if (  > 0) {
            
        } else {
            
            count ++;
        }
    }
}
```

The shell maintains a counter-- **count**-- to count how many new commands are executed by the shell. However, engineers find that the count doesn't work as planned. What causes the issue?

- A. It should use ++count
- B. The increment of the count variable should be placed after the while loop.
- C. The increment of the count variable should be placed before `exec()` in the child process section, i.e., within the if clause
- D. The increment of the count variable should be placed in the parent section, i.e., within the if clause.

Maximum marks: 8

10 Scheduling

Each blank is worth 2 points.

Completely Fair Scheduling (CFS) is the default scheduling algorithm used in Linux.

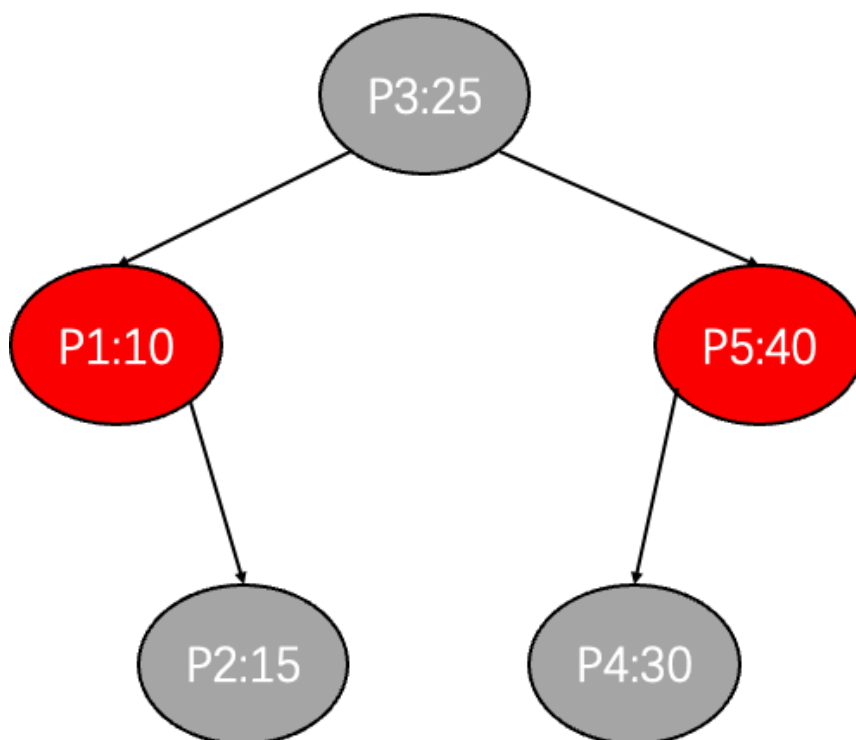
The objective of CFS is to guarantee (a metric) among all running processes.

Which property does CFS use to determine the schedule of processes or threads?

In the following, "the property" refers to the one answered in this question.

Which data structure does CFS use to implement efficient scheduling?

Each node in the data structure stores the property.



Px:Y : Px is the process number, while Y denotes the property of Px.

The above figure shows five processes currently running on the system. Which of the processes will be scheduled next?

If a process with a property value of 6 resumes after an I/O request, CFS will adjust its property. What will the new property value be based on the current status?

Maximum marks: 10

11 Paging

Each blank is worth 2 points.

The OS uses paging to manage virtual memory. The system has a 64-bit CPU and a multi-level paging.

Virtual address: 39 bits.

Page size: 4KB

Page table structure: 3 levels

Each level uses 9 bits of the virtual page number

Split the virtual address 0xFFFFDEADBEEF into three level page directory indices and an offset. The least significant bits are used for address translation. **Please answer them in hexadecimal numbers.**

L3	L2	L1	Offset

The following are some page-to-frame mappings.

Page number	physical frame
0x1DE	0x3000
0x0DB	0x12345
0x0F5	0x2000
0x1FF	0x1000
0xFFF	0x23BF
0xFDE	0x5ABC
0xADB	0x31CD

What is the physical address of 0xFFFFDEADBEEF?

Answer it in hexadecimal number.

Maximum marks: 10

12 Concurrency

Each blank is worth 2 points.

The system implements a ticket dispenser application for a bank.

There is one service desk and 10 seats available for consumers.

If a seat is available, a new consumer can take a ticket and wait for service.

The application involves two types of threads:

Bank staff thread: Processes one consumer's request at a time.

Customer threads: Take tickets and wait to be served at the desk.

Four semaphores are used to implement this application and successfully synchronize information among the threads.

Please fill in the following blanks to complete the code snippet.

1. Initialize three semaphores, empty, full, and mutex.

2. Add three semaphore operations in the bank_staff thread to manipulate three semaphores. Please ensure the correct order of the three operations.

semaphore empty = ; // Tracks available seats (initially all seats are empty)

semaphore mutex = ; // Ensures mutual exclusion when accessing the ticket dispenser

semaphore full = ; // Tracks the number of occupied seats by waiting customers

semaphore service = 0; // Indicates when a customer is ready to be served. Assume that the bank only has only one staff

```
Thread customer {
    sem_wait(empty);
    sem_wait(mutex);
    get_a_ticket();
    sem_post(mutex);
    sem_post(full);
    sem_wait(service);
    Be_served();
}
```

```
Thread bank_staff {
    while{True} {
         ;
         ;
         ;
        serve_customer();
    }
}
```

Maximum marks: 12

13 I/O devices

The ticket dispenser application discussed in the previous question uses a small printer (I/O device) to print tickets. Before sending data to the printer, the application must check its availability. What are the two methods to check the status of an I/O device? Please explain how they work. **(2 points)**

Fill in your answer here

If printing tickets is slow, which method should the ticket dispenser application use to check the status of the printer? Please explain why. **(1 points)**

Fill in your answer here

Maximum marks: 3

14 Comments

You can write some comments for your solutions here; these will not be graded.

Fill in your answer here

Maximum marks: 0